Derived or Observed: A Duty Written From First Principles Covers the Hazards It Can Derive and Misses the Ones Only Observation Reveals
A Reproducible Single-Turn Assay of Procedural-Specification Format and Hazard Coverage

Sophie D. Neilson
Independent Researcher
ORCID: 0009-0001-2430-1743

September 2026

Abstract

A procedure can be written in more than one form. We compare two forms of a *duty* (a gated procedure that governs a class of work) for repairing a confirmed software defect. The *annotated* form is written by reading a hazard registry and citing each entry it guards against, so the finished procedure depends on that registry to be understood. The *cartographer* form is written from first-principles analysis of the task, cites nothing, and is self-sufficient: an executor needs no external reference to follow it. We ask which hazards each form's procedure actually prevents. A local open-weight model (Qwen3.8-27B) writes the procedure in a single completion over a bare inference endpoint, and a blind judge scores, for each of ten pre-registered hazards, whether the procedure contains a step that would prevent it. The hazards are split in advance into seven that are derivable from the task and three that are not: a routing obligation upstream of the task, a session-close recording obligation, and a durability distinction learnable only from accumulated observation. Given the registry and told to write one gate per entry, the annotated form covers every hazard (210 of 210 derivable, 90 of 90 non-derivable); this is a manipulation check that the model follows the method, not a discovered effect. The two informative contrasts both involve the first-principles cartographer form: one against the no-method baseline, one against the annotated ceiling. The cartographer form covers the derivable hazards at a per-duty rate indistinguishable from a model given no method at all (0.64 versus 0.60; permutation p=0.44): writing from first principles buys self-sufficiency, not coverage. And it covers the non-derivable hazards almost never: 1 of 30 duties reaches any of them, against 30 of 30 for the annotated form (Fisher $p=5\times10^{-16}$). The pre-registered condition-by-class interaction, tested with the duty as the unit, is decisive (permutation $p<10^{-4}$). The cartographer form's distinctive property is self-sufficiency: it cites no registry entry in any run, where the annotated form cites all ten in every run. We then run a fix the form's proposers specified but never executed: a meta-taboo scan ("taboo" is the program's internal word for a hazard) that names two hazard classes, routing and session-close. It recovers exactly those two (routing 0 to 27 of 30; session-close 1 to 25 of 30) and not the third, the pure observation-only durability hazard it does not name (0 to 0 of 30). A blind rater judged coverage and an independent rater agreed (Cohen's $\kappa=0.85$); the direction is also corroborated without any judge, as the cartographer form never mentions routing across 30 runs and the scan form always does. Naming a class of hazard lets first-principles derivation reach it; a hazard whose badness is only apparent from accumulated observation, and that belongs to no named class, stays out of reach. The finding is a direction-and-rate result on one defect specimen and one model; the materials, runs, and scoring are public and reproduce on a 24 GB GPU with no paid API.

1 Introduction

Two people can write the same procedure in different forms. One writes by consulting a registry of known hazards and citing each one the procedure guards against; the reader of that procedure must know the registry to understand it. The other writes from first principles, analyzing the task and encoding each step so it stands on its own; the reader needs nothing but the procedure. The second form is more portable, since it carries its own justification, and a research program that writes machine-followed procedures (*duties*) prefers it for that reason. This paper asks what the portable form costs. A procedure written without the registry can only guard against the hazards its author can derive. If some hazards are not derivable from the task, if their danger is apparent only from having seen them happen, the self-sufficient form has no way to reach them.

We make this concrete with a repair-duty: a duty governing how an office repairs a confirmed software defect at a known locus. We hand a local open-weight model one of two writing methods and a repair task, and it emits the repair-duty in a single completion. We then score, for each of ten pre-registered hazards, whether the produced duty contains a gate that would prevent it. The ten hazards are pre-split into those derivable from the repair act and those that are not. The question is whether the form of the writing method decides which hazards the produced duty covers, and whether that decision falls along the derivable/not-derivable line.

This reformulates an earlier, unpublished two-run study in the same program, which reported the pattern from a single pair of hand-written procedures and proposed but never ran a fix. We restate it as a quantified, pre-registered, reproducible assay and run the fix.

1.1 Related Work

Whether the structure of a written specification changes how a model follows it is an active question for coding agents. Gloaguen et al. (2026) evaluate repository-level context files and find that an imperative instruction is followed while descriptive repository overviews add little. McMillan (2026) runs a factorial study of file-structure variables in coding-agent configuration files. Geng et al. (2026) measure the pragmatic influence of instruction wording, and Pecher et al. (2026) show classification behavior shifts with prompt underspecification. Our contribution is narrower and orthogonal to these: we hold the task fixed and vary only whether the procedure-writing method reads a hazard registry, and we measure hazard coverage of the produced procedure rather than task accuracy. We searched Google Scholar and arXiv (September 2026) for work on whether the form of a procedural specification determines which hazards it covers, as distinct from whether it is followed; we found none directly on point, and the coding-agent configuration studies above are the nearest neighbors. The duty, taboo, and cartographer/annotated vocabulary is internal to this research program; "taboo" is the program's internal word for a hazard. The study reformulated here is earlier, unpublished internal work in the same program; its materials are reformulated and published with this paper in the study directory named in Data Availability.

2 Materials and Method

2.1 Terminology

This study uses a small set of coined terms from the research program that writes machine-followed procedures. Table 1 defines them; the abstract and every section use these names.

Table 1: The study's coined terms.

Term	What it is
Duty	A gated procedure that governs a class of work. The artifact the model writes here is a *repair-duty*: a sequence of gates an executor passes through, each stating what it verifies and what completing it requires.
Gate	A single checkable step inside a duty. A hazard is *covered* when the duty holds a gate whose action would prevent it.
Hazard	A divergence point a repair can cross: one of the ten pre-registered failure modes the produced duty is scored against. ("Taboo" is the program's internal word for a hazard.)
Divergence point	The specific way a repair goes wrong that a hazard names, and that a gate must block.
Registry (the canon)	The list of the ten hazards. The annotated method reads it; the cartographer method does not.
Coverage	Whether a produced duty holds a gate whose action would prevent a given hazard's divergence point. The primary judged measure. Naming a hazard is not coverage; the gate's action is.
Derivable / non-derivable hazard	The pre-registered split. Seven hazards are derivable by analyzing the repair act; three are not: an upstream routing obligation, a session-close recording obligation, and an observation-only durability distinction.
Baseline	A writing condition: the repair task and defect only, no method and no registry. The floor.
Annotated form	A writing method that reads the registry and writes one gate per hazard, citing each entry. Its duties depend on the registry to be understood.
Cartographer form	A writing method that derives the failure modes from first principles, cites nothing, and is self-sufficient: an executor needs no external reference.
Meta-taboo scan (cartographer + scan)	The cartographer method plus a pass that names two hazard classes sitting outside the act, routing and session-close, and directs a gate for each.

2.2 The reformulation

The earlier study used a multi-turn agent that read files and wrote a procedure to disk. That subject cannot run on the reproducible endpoint this program requires, so the assay is recast as a single-turn generation: the model receives a writing method and the repair task, and emits the complete repair-duty as its one reply. There is no harness, no tool use, and no second turn; the produced text is the artifact scored. This preserves the question (does the writing method's form decide hazard coverage?) while fitting an endpoint anyone can reproduce.

2.3 Conditions and materials

The independent variable is the writing method. Information is not matched across conditions, by design: the two forms are defined by their relationship to the hazard registry, so the annotated method carries the registry and the cartographer method does not. This asymmetry is the object of study, not a confound. A hazard whose danger is only apparent from accumulated observation cannot be derived from the task; withholding the registry from the cartographer condition is what makes "does first-principles derivation reach this hazard?" a real question rather than "will the model copy a list it can see?" A pilot with the registry placed in the shared prompt

confirmed the latter: the model copied the observation-only hazards and the effect vanished.

- **baseline**: the repair task and the defect only, no writing method and no registry. The floor.
- **annotated**: the annotated method, carrying the ten-hazard registry: read it, open with a hazard table, write one gate per hazard, and cite the registry entry in each gate body.
- **cartographer**: the cartographer method, no registry: derive the failure modes from the repair act, encode each as a self-sufficient gate, cite nothing.
- **cartographer+scan**: the cartographer method plus a meta-taboo scan that names two hazard classes, routing (upstream of the act) and session-close obligations (after the act), and directs a structural gate for each.

The shared task names one defect: `calculate_average` divides by `len(values)` and raises on an empty list, locus confirmed. The writing methods and the registry are reproduced in Appendix A.

2.4 The ten hazards and the pre-registered split

Each hazard names a divergence point a repair can cross. Seven are derivable from analyzing the repair act (a confirmed locus before editing; a declared pass criterion; a scope boundary; fixing the canonical source not a call site; a revised root cause before a second attempt; recorded completion evidence; verification by observed result not claim). Three are not: a *routing* obligation upstream of the act (confirm a repair, not another task, is what is needed); a *session-close* obligation (record a constraint discovered during the repair before closing); and a *durability* distinction (advisory versus structural fixes), whose badness is learnable only from accumulated observation. The split, the divergence points, and which hazards the scan names are fixed before the run in `studies/cartographer-duty-format/taboo_set.json`.

2.5 Model and Sampling

The subject is Qwen3.8-27B (Q4_K_M GGUF) served over a bare `llama.cpp` raw completion endpoint (no chat template, no tools) with thinking pinned off in the published wrapper. The complete sampler chain is declared in the study definition and sent with every request: temperature 0.8; `min_p` 0.05 as the sole live truncation stage; `top_k`, `top_p`, `typical_p`, `top_n_sigma`, all penalties, and the XTC and DRY stages off; one seed per replicate (1 through 30); a 4096-token cap. Each condition ran 30 replicates. The server ran on a GPU (RTX 3090); recorded throughput held between 37.9 and 40.8 tokens per second across all 120 runs, with no step down to a CPU rate, and no response was truncated at the cap (the longest was 3806 predicted tokens). The served build, read from the endpoint's `props` at the session, was `b9445-af6528e6d`; the per-row build field was empty in this run's record, a recorded gap noted here rather than inferred. The probe image digest and the substrate probe are in the run record. The repository was on commit `56eb443` with a dirty working tree when the run launched, disclosed here; the committed materials and instrument are in the repository directory named in Data Availability.

2.6 Scoring

Two measurements. The first is deterministic: a script counts how many of the ten registry slugs each produced duty cites (`slug_c2.py`), which needs no judge. The second is coverage: for each duty and each hazard, does the duty contain a gate whose action would prevent that hazard's divergence point? Naming a hazard is not coverage; the gate's action is. Coverage was judged by Claude (Opus 4.8), blind to condition: the 120 duties were shuffled and stripped of

their condition labels into an opaque-id set before judging, and unblinded only for analysis. An independent second rater, also blind and given a clean rubric with no worked example, re-scored a stratified 40-duty sample; agreement is reported in Section 3.

2.7 Agents and Models

Role	Model	Configuration	Date
Subject	Qwen3.8-27B	Q4_K_M GGUF, `llama.cpp` build `b9445-af6528e6d` (per `props`), raw completion, thinking off, GPU (RTX 3090), context 32768.	2026-09-12
Slug scorer (C2)	Deterministic rule	`slug_c2.py`; exact registry-slug match on the response text; no judgment.	2026-09-12
Coverage judge	Claude Opus 4.8	Blind to condition (shuffled, de-labelled set); four batches over the 120 duties. Not a treatment arm.	2026-09-12
Second rater	Claude Opus 4.8	Independent session, blind, clean rubric with no worked example; stratified 40-duty sample. Same family as the primary judge (see Limitations).	2026-09-12
Author and operator	Claude Opus 4.8	Authored the materials, ran the study, wrote the coverage rubric. Not a treatment subject.	2026-09-12

Table 2: Agents and models. The subject is the only treatment arm and is a local open-weight model; Claude is used only to judge and to operate, never as a treatment condition. All dates are UTC (the run record stores UTC; the run ran on 2026-09-12, 01:07–02:35Z).

3 Results

3.1 Slug citation is a clean, deterministic separation

The annotated method cites all ten registry slugs in every one of its 30 runs (mean 10.0 slugs). The baseline, cartographer, and cartographer+scan conditions cite none, in any run. The self-sufficiency the cartographer form is written for holds exactly: its produced duties carry no dependency on the registry, where the annotated form's carry all of it.

3.2 Coverage splits along the derivable line

Table 3 gives coverage by hazard class as covered/judged calls, and the per-duty means used for the tests below. The annotated method covers everything (210 of 210, 90 of 90). This is a manipulation check, not a discovered effect: the annotated method is handed the exact ten-hazard registry and told to write one gate per entry, so full coverage is a ceiling by construction, and it confirms only that the model followed the method. The scientific weight is in the two contrasts involving the first-principles cartographer form: against the no-method baseline and against the annotated ceiling.

All inferential tests below take the duty as the unit of analysis (n=30 per condition), because the 3 or 7 hazard calls within one produced duty are not independent; the covered/judged counts in the table are reported as descriptive rates, not as independent trials.

First, the cartographer method's derivable coverage is not distinguishable from the no-method baseline: per-duty rate 0.64 versus 0.60 (permutation test on per-duty rates, p=0.44). Writing from first principles does not raise coverage over giving the model no method at all; what

it changes is the dependency, not the coverage. This is a failure to reject, not a demonstration of equivalence: the per-duty rates overlap and a small real difference is not excluded. Second, the two methods diverge almost completely on the non-derivable hazards: every one of the 30 annotated duties reaches all three, and only 1 of the 30 cartographer duties reaches any (Fisher exact on duties, $p=5\times10^{-16}$). The pre-registered condition-by-class interaction is the difference between these: the annotated advantage over cartographer is 0.36 on derivable hazards and 0.99 on non-derivable ones. Tested at the duty level as the difference-in-differences (each duty's derivable rate minus its non-derivable rate, annotated versus cartographer), the interaction is decisive: the annotated per-duty difference is 0.00 and the cartographer's is +0.63 (permutation test, $p<10^{-4}$; no resample of 100,000 reached the observed separation).

Within the derivable set the cartographer method is heterogeneous, so the pooled 0.64 flatters it: per-hazard coverage runs from 2 of 30 (revising the root cause before a second attempt) to 30 of 30 (requiring a confirmed locus), and two derivable hazards are missed badly (Section 3.5).

Condition	**Derivable (7 hazards)**	**Non-derivable (3 hazards)**
baseline	127/210 (0.60)	4/90 (0.04)
annotated	210/210 (1.00)	90/90 (1.00)
cartographer	134/210 (0.64)	1/90 (0.01)
cartographer+scan	133/210 (0.63)	52/90 (0.58)

Table 3: Hazard coverage by class, as covered/judged calls with the per-duty mean rate in parentheses, over 30 duties per condition and 7 or 3 hazards. Derivable hazards are those a first-principles analysis of the repair act can reach; non-derivable are the routing, session-close, and durability hazards. The annotated row is a construction ceiling (see text); inferential tests use the per-duty rates.

3.3 The scan recovers the classes it names

The meta-taboo scan names two hazard classes. Table 4 shows it recovers exactly those two and not the third. Routing coverage rises from 0 of 30 to 27 of 30, and session-close from 1 of 30 to 25 of 30. The durability hazard, which the scan does not name and which is learnable only from observation, stays at 0 of 30. Naming a class of hazard lets first-principles derivation reach it; the hazard that belongs to no named class remains unreachable.

Hazard	**Class**	**Scan names it**	**cartographer**	**+scan**	**Fisher p**
routing	meta/routing	yes	0/30	27/30	9.2×10^{-14}
session-close	session-close	yes	1/30	25/30	1.2×10^{-10}
durability	observation-only	no	0/30	0/30	1.0

Table 4: Cartographer versus cartographer+scan on the three non-derivable hazards. The scan recovers the two classes it names and not the one it does not.

3.4 The direction holds without a judge

Coverage is a judged measurement, so we corroborate its direction with a deterministic keyword-presence check that needs no rater: does a produced duty even mention the subject of each non-derivable hazard? The pattern matches the judged coverage. No cartographer duty mentions routing (0 of 30) and every scan duty does (30 of 30), tracking the judged 0-to-27 recovery. The advisory or durability vocabulary appears in 2 of 30 cartographer duties and 3 of 30 scan duties, tracking the judged non-recovery of the durability hazard. The annotated duties mention all

three subjects in every run (30 of 30 each). Mentioning a subject is weaker than covering it with a gate, so these counts sit at or above the judged coverage rates; their value is that the divergence between the forms is visible in the raw text without any judgment.

3.5 Two nominally derivable hazards behave like the hard ones

The derivable/not-derivable split predicts the broad pattern, but it is not perfectly clean. Two hazards labelled derivable are missed by the cartographer method almost as often as the non-derivable ones: fixing the canonical source rather than a call site (7 of 30) and revising the root cause before a second attempt (2 of 30). Only the annotated method, carrying the registry, covers them reliably (30 of 30 each). These two sit closer to the hard end than their label implies: they are derivable in principle but subtle enough that a from-scratch analysis usually does not surface them. We report the split as pre-registered and flag these two rather than re-labelling them after the fact.

3.6 Inter-rater agreement

The independent second rater, blind and given a clean rubric with no worked example, re-scored a stratified 40-duty sample (10 per condition). Agreement with the primary judge was 372 of 400 hazard calls (0.93; Cohen's $\kappa = 0.85$). The disagreements concentrate on one borderline hazard, whether recorded completion evidence is required (29 of 40); the hazards that carry the finding agree strongly, including the observation-only durability hazard at 40 of 40 and routing at 39 of 40. Because the second rater saw no worked example, the agreement indicates the primary coverage calls are not an artifact of the worked example in the primary rubric.

4 Discussion

The result draws a line under a portable procedure. A duty written from first principles, needing no external reference, is exactly as good as its author's ability to derive hazards from the task, and no better, on this evidence, than a model given no method at all. Its value is that it carries no dependency, not that it prevents more. The hazards it misses are the ones a task cannot teach: an upstream routing check, a session-close obligation, and a durability distinction that only accumulated observation reveals. The annotated form reaches all of them because it copies them from a registry that already remembers them.

The scan result sharpens this. A hazard that a first-principles author misses is not always unreachable; it is unreachable until it is named. Naming the class ("check for routing hazards," "check for session-close obligations") gives the derivation a place to look, and coverage of those classes jumps from near zero to most runs. What naming does not rescue is the hazard whose content, not just its existence, must be observed: the advisory-versus-structural durability distinction stays at zero even with the scan running, because knowing to look for durability hazards does not tell you what makes an advisory fix fragile. That is the residue that a self-sufficient format cannot, in principle, carry.

4.1 Limitations

One defect specimen, one model, single-turn generation. This is a direction-and-rate finding on the `calculate_average` empty-list defect as written for Qwen3.8-27B; the derivable/not-derivable boundary may fall differently for another domain, another model, or a multi-turn agent that can consult a registry by choice rather than having it withheld. The information asymmetry between the forms is deliberate and is discussed in the Method; a consequence is that the non-derivable coverage gap is partly expected, since a hazard that must be observed cannot

be derived, so the informative quantities are how much of the derivable set the cartographer form recovers (most) and whether naming a class recovers the rest (for two of three, yes).

The coverage result is judge-dependent, and both the coverage judge and the second rater are Claude (Opus 4.8). This is a real limit: two instances of one architecture can agree because they share a blind spot, so the κ=0.85 is within-family agreement exposed to correlated error, not cross-architecture confirmation. What does not depend on a judge is narrower than the whole finding: the deterministic slug count establishes only the self-sufficiency of the cartographer form (it cites no registry entry), not the coverage rates. The coverage rates (the 0.64, the 0.01, the scan recovery) rest on the Claude judge. Two things bound the risk. The primary judge saw a worked example in its rubric that could have anchored it; the second rater was given no such example, and the two agree, so the calls are not an artifact of that example. And the direction is corroborated by the rater-free keyword-presence check (Section 3): the routing gap and the durability non-recovery are visible in the raw text with no judgment at all. A local cross-architecture coverage rater remains future work.

Two smaller items. The subject is named by its served GGUF alias, `Qwen3.8-27B-Q4_K_M.gguf`; the endpoint attests that filename, not an upstream release identity, so the model claim is only as good as the local artifact name. And of the earlier study's added criteria, C5 (a coverage-map slug leak in the cartographer output) is answered by the slug count: no cartographer or scan duty cited any slug (0 of 30 each), so there is no leak. C8 (a patchover check for proxy completion conditions) requires a separate judgment we did not perform and is retired here, as C7 (an instrument-read carve-out, a self-reference gate specific to the original multi-turn duty-writing setup) was retired as not applicable to single-turn generation.

The pre-registered derivable split is imperfect for two hazards, disclosed above and not re-labelled.

5 Conclusion

A self-sufficient procedure covers the hazards its author can derive from the task and misses the ones that can only be observed. Writing from first principles does not, on this evidence, prevent more failures than no method at all; it removes the dependency on a registry, and with it the reach to any hazard the registry alone remembers. Naming a class of hazard restores the reach for hazards whose existence is the only thing that must be supplied; it does not restore reach for a hazard whose content must be observed. A portable format and complete hazard coverage are, in this setting, two different things.

5.1 Data Availability

The complete study, comprising the writing methods, the shared task and defect, the pre-registered hazard set and split, the design and pre-registration documents, the run records with per-row provenance, the produced duties, the blind judging set and map, the deterministic slug scorer, the coverage grid, and the analysis script, is available at `https://github.com/sophdn/corpos-lab` under the `studies/cartographer-duty-format/` directory. The instrument change that added the three format conditions is in the same repository under `internal/`.

References

Geng, Y., Abend, O., Hovy, E., and Frermann, L. (2026). Measuring Pragmatic Influence in Large Language Model Instructions. *arXiv preprint arXiv:2602.21223*.

Gloaguen, T., Mündler, N., Müller, M., Raychev, V., and Vechev, M. (2026). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? *arXiv preprint arXiv:2602.11988*.

McMillan, D. (2026). Instruction Adherence in Coding Agent Configuration Files: A Factorial Study of Four File-Structure Variables. *arXiv preprint arXiv:2605.10039*.

Pecher, B., Spiegel, M., Belanec, R., and Cegin, J. (2026). Revisiting Prompt Sensitivity in Large Language Models for Text Classification: The Role of Prompt Underspecification. *arXiv preprint arXiv:2602.04297*.

A Materials

The shared task and defect, the three writing methods, and the ten-hazard registry carried by the annotated method are reproduced below, as handed to the model. The writing-method prose is reproduced verbatim; the registry, a Markdown table in the source, is rendered here as Table 5 with the same ten entries and text. The pre-registered derivable/non-derivable split and the scan-target flags are in `taboo_set.json`, and the coverage judging brief in `scores/JUDGE_BRIEF.md`, both under the study directory named in Data Availability.

A.1 Shared task and defect

````
# Task: write a repair-duty

Write a duty named `repair-duty` that governs how any office of the Agentic
Order repairs a confirmed software defect with a known locus. A duty is a Rule
made navigable: a sequence of gates an executing office passes through, where
each gate states what it verifies and what completing it requires. The duty you
write will be handed to a future executor who has only the duty text.

Use the following defect as the domain anchor. It is context, not the object of
the task; do not fix it yourself.

```python
# calculator.py
def calculate_average(values: list) -> float:
    return sum(values) / len(values)
```

`calculate_average([])` raises `ZeroDivisionError`. The locus is confirmed at
`calculator.py:2`.

## Output format

You are running as a single completion. There is no harness, no tools, no
filesystem, no shell, and no second turn. You cannot read files, run code, or
consult anything beyond this prompt. Produce the complete `repair-duty` document
now, as your entire reply, in Markdown. Write the duty itself, not a plan to
write it.
````

A.2 Annotated method

The annotated method reads the registry and writes one gate per entry. Its prose is reproduced verbatim here; the registry it carries is Table 5.

```
# Method for writing a duty: the annotated form

Write the duty by mapping the canon onto gates. This is the standard form, and
it depends on the canon: you read the taboo registry and encode each entry that
applies as a gate.

## The taboo registry (the canon)
```

```
Each taboo names a divergence point a repair can cross.
```

Taboo	Divergence point
`triggers-routing`	Repair execution begins before routing is confirmed; a misrouted prompt runs a repair where an investigation was needed, or the reverse.
`explicit-prerequisite-gate`	Repair begins without a confirmed locus record naming file, line, and failure mode.
`investigation-fix-criterion-predeclaration`	A fix attempt begins before a passing criterion is declared, so the criterion can be fitted to whatever change was made.
`investigation-fix-scope-boundary`	A change touches lines the confirmed locus did not name; opportunistic adjacent edits break the anchor to root cause.
`fix-locus-identification`	The fix is applied to a call site or consumer, leaving the canonical source location unchanged.
`fix-attempt-root-cause-reassessment`	A second fix attempt follows a failed one without a revised root-cause citation.
`investigation-advisory-fix-durability`	An advisory fix (comment, docstring, warning) is accepted without a durability check confirming a caller cannot bypass it. Advisory-versus-structural durability is a distinction learned from accumulated observation, not derived from the defect.
`completion-evidence-required`	The repair is declared complete without a co-located verification artifact.
`document-claim-verification`	Verification is by written claim rather than an observed test result or output.
`known-constraint-documentation`	An environmental constraint discovered during repair is not recorded before the session closes.

Table 5: The ten-hazard taboo registry carried by the annotated method, reproduced from `materials/annotated.md`.

```
## The method

1. **Read the registry above.** For every taboo whose divergence point a repair
   could cross, the duty must carry a gate that prevents it.
2. **Open with a Taboos table.** List each relevant taboo slug and one line
   saying why it applies to this duty.
3. **Write one gate per divergence point.** Each gate is stamped with the taboo
   slug it enforces, in the form `*Taboo: \`slug\`*`. State when the gate fires,
   when it does not, and the rejection clause that names the failure it blocks.
4. **Cite the slug in the gate body.** The gate names the canon entry it
   enforces so an executor can trace the gate to its taboo. The canon is a
   runtime dependency: an executor who does not know what a cited slug means
   consults the registry.
5. **Cover every registered taboo that applies.** Completeness is measured
   against the registry: a taboo with no gate is a hole.

Produce the duty in this annotated form. Cite the slugs.
```

A.3 Cartographer method

```
# Method for writing a duty: the cartographer form

Write the duty as a map an executor can follow without knowing the canon. The
executor who receives this duty has only its text; the duty must let them avoid
every divergence point without reading any registry at runtime.

1. **Derive the failure modes from first principles.** Analyze the repair act
   itself. Ask what an office doing this work could get wrong: where the act
   begins, what it must establish before it changes anything, what a correct
   change looks like, and how completion is confirmed. Build the gate list from
   that analysis of the act.
2. **Encode avoidance structurally.** Each gate states what it verifies and what
   completing it requires. A gate is a checkable step, not a description of a
   hazard.
3. **Cite no taboo slugs.** You have no registry; there is no canon to cite. Do
   not open with a taboo table. Each gate stands on its own words, derived from
```

```
   your analysis of the act. The duty must be self-sufficient: an executor
   needs no canon to run it.
4. **Group the gates into phases** that follow the shape of the work, from
   confirming the locus through to closing out the repair.

Produce the duty in this cartographer form. Derive the gates from the act; cite
no slugs.
```

A.4 Cartographer method with the meta-taboo scan

```
# Method for writing a duty: the cartographer form with a meta-taboo scan

Write the duty as a map an executor can follow without knowing the canon. The
executor who receives this duty has only its text; the duty must let them avoid
every divergence point without reading any registry at runtime.

1. **Derive the failure modes from first principles.** Analyze the repair act
   itself. Ask what an office doing this work could get wrong: where the act
   begins, what it must establish before it changes anything, what a correct
   change looks like, and how completion is confirmed. Build the gate list from
   that analysis of the act.
2. **Encode avoidance structurally.** Each gate states what it verifies and what
   completing it requires. A gate is a checkable step, not a description of a
   hazard.
3. **Cite no taboo slugs.** You have no registry; there is no canon to cite. Do
   not open with a taboo table. Each gate stands on its own words, derived from
   your analysis of the act. The duty must be self-sufficient: an executor
   needs no canon to run it.
4. **Group the gates into phases** that follow the shape of the work, from
   confirming the locus through to closing out the repair.
5. **Run a meta-taboo scan.** First-principles analysis of the repair act
   surfaces the hazards inside the act, but two classes of hazard sit outside
   it and will not surface from that analysis alone. Scan explicitly for both
   and add a structural gate for each that applies:
   - **Routing-class hazards** - obligations that sit *upstream* of the act.
     Before the repair runs, was it confirmed that a repair, and not some other
     work, is what this situation calls for?
   - **Session-close-obligation hazards** - obligations that fire *after* the
     act, on session close rather than on the repair itself. Was anything
     discovered during the repair that must be recorded before the session ends?
   Encode these structurally as well; still cite no slugs.

Produce the duty in this cartographer form with the meta-taboo scan. Derive the
gates from the act, run the scan for upstream and session-close hazards, and
cite no slugs.
```